# ASTEROIDS - FdV
# Por: Bruno Villar

RESUMEN DEL JUEGO

Se trata de un juego en 2D en el que el jugador controlará una nave con las teclas de movimiento o las teclas ”wasd” con unos controles basados en los controles de tanque. El objetivo del juego es conseguir la máxima cantidad de puntos posible disparando a unos asteroides que van cayendo del cielo y que se dividen en 2 al dispararlos. El juego posee menú de pascua al cual se puede acceder mediante botones en la propia pantalla o mediante la tecla Esc.

ESCENAS

Consta en si de tan solo una escena principal en la que ocurre todo, el fondo es de espacio, y la pantalla de pausa forma parte de dicha escena también. Es una escena 3D de la cual se ha utilizado tan solo el componente 2D (x e y) ya que la z en este caso no era necesaria para poder hacer el juego.

ASSETS Y GAMEOBJECTS

- **Player**: Nave espacial con RigidBody, Collider y el script Player.cs
- Spawner: Objeto hijo vacío que está en el cañón de la nave para indicar el punto de salida de los proyectiles
- **Space**: Objeto que sirve de fondo
- **AsteroidPrefab**: Prefab base para los meteoritos principales con el script Asteroid.cs
- **MiniAsteroidPrefab**: Sub-prefab generado al destruir un asteroide grande.
- **EnemySpawner**: Objeto con el script EnemySpawner.cs que controla la tasa de aparición de asteroides.
- **Canvas** UI: Interfaz gráfica que muestra el contador de puntos (Text) y el menú de pausa (Panel).

MECÁNICAS DE JUEGO

El game loop del juego es el siguiente:

1. El jugador aparece en un lado de la pantalla.
2. Los asteroides se generan continuamente desde arriba (cada vez generando más).
3. El jugador dispara balas para destruir asteroides y con cada asteroide destruido se suma un punto al marcador de puntuación (al disparar a un asteroide, se divide en dos mini-asteroides que salen en ángulos opuestos)
4. Si un asteroide colisiona con el jugador, la puntuación se reinicia a 0 y se recarga la escena.

SCRIPTS

- **Player.cs:** Aplica fuerzas físicas al Rigidbody para el movimiento de la nave.
- **ObjectPool.cs:** Implementa el diseño de “piscina de objetos” al juego para optimizar el rendimiento.
- Bullet.cs: Controla el movimiento en línea recta de la bala, la desactivación según su tiempo de vida y la detección de colisiones.
- **EnemySpawner.cs**: Genera constantemente asteroides en posiciones horizontales aleatorias encima de lo que ve el jugador (no se ve el momento en el que se generan)
- **Asteroid.cs**: Gestiona la destrucción y la separación de los meteoritos

INTERACCIÓN

Vuelo

- Rotación: Teclas A y D
- Propulsión: Teclas W y S
- Límites de la pantalla: Cuando la nave supera unos límites establecidos en la x y la y de la pantalla pasa al borde opuesto lo que da una sensación de espacio infinito

Disparo y destrucción

- Creación y disparo de proyectiles: Tecla espacio
- Impacto: Al colisionar una bala con un asteroide (detectado con el tag Enemy y el collider) se añade un punto y se destruye el asteroide

Interfaz UI y muerte

- Pausa: Se activa un menú de pausa con la tecla Esc o con el botón de la pantalla. Se desactiva de la misma forma.
- Muerte: La colisión entre la nave y un asteroide resetea los puntos y devuelve los valores a su posición inicial.

# SCRIPTS (no puedo hacerlo más grande porque ponía máx 4 páginas)

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class Player : MonoBehaviour
{
    public float velocidadEmpuje = 200f;
    public float velocidadRotacion = 120f;

    public GameObject spawner, bulletPrefab;

    private Rigidbody rigid;

    public float xBorderLimit = 10f;
    public float yBorderLimit = 6f;

    public static int SCORE = 0;

    public GameObject panel;

    public ObjectPool miPool;

    void Start()
    {
        SCORE = 0;
        Time.timeScale = 1;
        rigid = GetComponent<Rigidbody>();
    }

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            if (Time.timeScale == 1)
            {
                pause();
            }
            else
            {
                resume();
            }
        }

        if (Time.timeScale == 0) return;

        float rotacion = Input.GetAxis("Horizontal") * Time.deltaTime;
        float empuje = Input.GetAxis("Vertical") * Time.deltaTime;

        Vector3 direccionEmpuje = transform.right;

        rigid.AddForce(direccionEmpuje * empuje * velocidadEmpuje);

        transform.Rotate(Vector3.forward, -rotacion * velocidadRotacion);

        var newPos = transform.position;

        if (newPos.x > xBorderLimit)
            newPos.x = -xBorderLimit + 1;
        else if (newPos.x < -xBorderLimit)
            newPos.x = xBorderLimit - 1;
        else if (newPos.y > yBorderLimit)
            newPos.y = -yBorderLimit + 1;
        else if (newPos.y < -yBorderLimit)
            newPos.y = yBorderLimit - 1;

        transform.position = newPos;

        if (Input.GetKeyDown(KeyCode.Space))
{
    if (miPool != null && spawner != null)
    {
        GameObject bullet = miPool.GetPooledObject();
        if (bullet != null)
        {
            bullet.transform.position = spawner.transform.position;
            bullet.transform.rotation = spawner.transform.rotation;

            Bullet balaScript = bullet.GetComponent<Bullet>();
            if (balaScript != null)
            {
                balaScript.targetVector = transform.right;
            }

            bullet.SetActive(true);
        }
    }
}
}

    private void OnCollisionEnter(Collision collision)
    {
        if (collision.gameObject.tag == "Enemy")
        {
            SCORE = 0;
            SceneManager.LoadScene(SceneManager.GetActiveScene().name);
        }
        else
        {
            Debug.Log("He colisionado con otra cosa...");
        }
    }

    public void pause()
    {
        Time.timeScale = 0;
        panel.SetActive(true);
    }

    public void resume()
    {
        Time.timeScale = 1;
        panel.SetActive(false);
    }
}

using UnityEngine;
using UnityEngine.UI;

public class Bullet : MonoBehaviour
{
    public float velocidad = 10f;
    public float tiempoVida = 3f;
    public Vector3 targetVector;

    private void OnEnable()
    {
        Invoke(nameof(DesactivarBala), tiempoVida);
    }

    private void OnDisable()
    {
        CancelInvoke(nameof(DesactivarBala));
    }

    private void DesactivarBala()
    {
        gameObject.SetActive(false);
    }

    // Start is called once before the first execution of Update after the MonoBehaviour is created
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

        transform.position += targetVector * velocidad * Time.deltaTime;
    }

    private void OnCollisionEnter(Collision collision){


        if(collision.gameObject.tag == "Enemy"){

            Collider enemyCollider = collision.gameObject.GetComponent<Collider>();

            if (enemyCollider != null && enemyCollider.enabled) // Hago esto para que dos balas dando a un meteorito no consigan 2 puntos, tan solo 1
            {
                enemyCollider.enabled = false;
                IncreaseScore();
                Destroy(collision.gameObject);
                gameObject.SetActive(false);
            }
            else{
```

```
                gameObject.SetActive(false);
            }


        }
    }

    private void IncreaseScore(){

        Player.SCORE++;
        UpdateScoreText();

    }

    private void UpdateScoreText(){

        GameObject puntos = GameObject.FindGameObjectWithTag("UI");
            puntos.GetComponent<Text>().text = "Puntos : " + Player.SCORE;



    }
}

using UnityEngine;
using System.Collections.Generic;

public class ObjectPool : MonoBehaviour
{
    public List<GameObject> pooledObjects;
    public GameObject objectToPool;
    public int amountToPool;

    void Start()
    {
        pooledObjects = new List<GameObject>();
        GameObject pooleado;
        for (int i = 0; i < amountToPool; i++)
        {
            pooleado = Instantiate(objectToPool);
            pooleado.SetActive(false);
            pooledObjects.Add(pooleado);
        }
    }

    public GameObject GetPooledObject()
    {
        for (int i = 0; i < pooledObjects.Count; i++)
        {
            if (!pooledObjects[i].activeInHierarchy)
            {
                return pooledObjects[i];
            }
        }
        GameObject nuevaBala = Instantiate(objectToPool);
        nuevaBala.SetActive(false);
        pooledObjects.Add(nuevaBala);
        return nuevaBala;
    }
}
using UnityEngine;

public class EnemySpawner : MonoBehaviour
{
    public GameObject asteroidPrefab;

    public float spawnRatePerMinute = 30;
    public float spawnRateIncrement = 1f;

    private float spawnTime = 0;

    public float limite;

    public float tiempoVida = 3f;

    // Update is called once per frame
    void Update()
    {
        if (Time.time > spawnTime){

            spawnTime = Time.time + 60/spawnRatePerMinute;

            spawnRatePerMinute += spawnRateIncrement;

            float rand = Random.Range(-limite, limite);

            Vector2 spawnPosition = new Vector2(rand, 8f);

            GameObject meteor = Instantiate(asteroidPrefab, spawnPosition, Quaternion.identity);

            Destroy(meteor, tiempoVida);
        }
    }


}

using UnityEngine;

public class Asteroid : MonoBehaviour
{
    public GameObject miniAsteroidPrefab;
    public float anguloGrados = 45f;
    public float velocidadMini = 5f;
    public bool esMini = false;

    private void OnCollisionEnter(Collision collision)
    {
        if (collision.gameObject.CompareTag("Bullet") || collision.gameObject.name.Contains("Bullet"))
        {

            if (!esMini && miniAsteroidPrefab != null)
            {
                Vector3 dirBala = Vector3.up;
                Bullet bala = collision.gameObject.GetComponent<Bullet>();

                if (bala != null && bala.targetVector != Vector3.zero)
                {
                    dirBala = bala.targetVector.normalized;
                }
                else
                {
                    dirBala = (transform.position - collision.transform.position).normalized;
                }

                float rad = anguloGrados * Mathf.Deg2Rad;

                Vector3 dirIzquierda = new Vector3(
                    dirBala.x * Mathf.Cos(rad) - dirBala.y * Mathf.Sin(rad),
                    dirBala.x * Mathf.Sin(rad) + dirBala.y * Mathf.Cos(rad),
                    0f
                );

                Vector3 dirDerecha = new Vector3(
                    dirBala.x * Mathf.Cos(rad) + dirBala.y * Mathf.Sin(rad),
                    -dirBala.x * Mathf.Sin(rad) + dirBala.y * Mathf.Cos(rad),
                    0f
                );

                CrearMiniAsteroide(dirIzquierda);
                CrearMiniAsteroide(dirDerecha);
            }

            Destroy(gameObject);
        }
    }

    private void CrearMiniAsteroide(Vector3 direccion)
    {
        GameObject mini = Instantiate(miniAsteroidPrefab, transform.position, Quaternion.identity);

        Rigidbody nuevo = mini.GetComponent<Rigidbody>();
        if (nuevo != null)
        {
            nuevo.linearVelocity = direccion * velocidadMini;
        }
    }
}
```